

Practical 3

Aim:

To implement a Word Count / Word Frequency application using Hadoop MapReduce for processing text files stored in HDFS, generating word-frequency statistics, identifying the top 10 most frequently occurring words, excluding common stop words, and analyzing the effect of different input dataset sizes on MapReduce execution time.

Problem definition:

A digital publishing company stores thousands of news articles and blog posts in a Hadoop Distributed File System (HDFS). The editorial team wants to identify the most frequently used words across their content to understand trending topics and optimize content recommendations. As a Big Data Engineer, you are required to develop a MapReduce application that processes the text data and generates word frequency statistics.

Objectives:

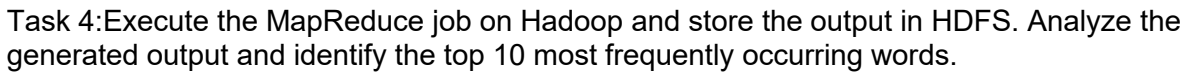
1. To configure the Hadoop environment and verify HDFS and YARN services.
2. To upload one or more text datasets into the Hadoop Distributed File System (HDFS).
3. To implement a Mapper that reads text data, extracts words, and generates intermediate key-value pairs in the form:
4. To implement a Reducer that groups identical words and calculates their total frequency.
5. To execute the Word Count application as a Hadoop MapReduce job.
6. To store and analyze the generated output in HDFS.
7. To identify the top 10 most frequently occurring words.
8. To exclude common stop words such as a, an, the, is, and are from the analysis.
9. To compare word-frequency results for multiple input files.
10. To execute the application using datasets of different sizes and compare their processing time and output.
11. To understand the advantages of distributed processing using Hadoop MapReduce compared with conventional sequential processing.

Observation / Output:

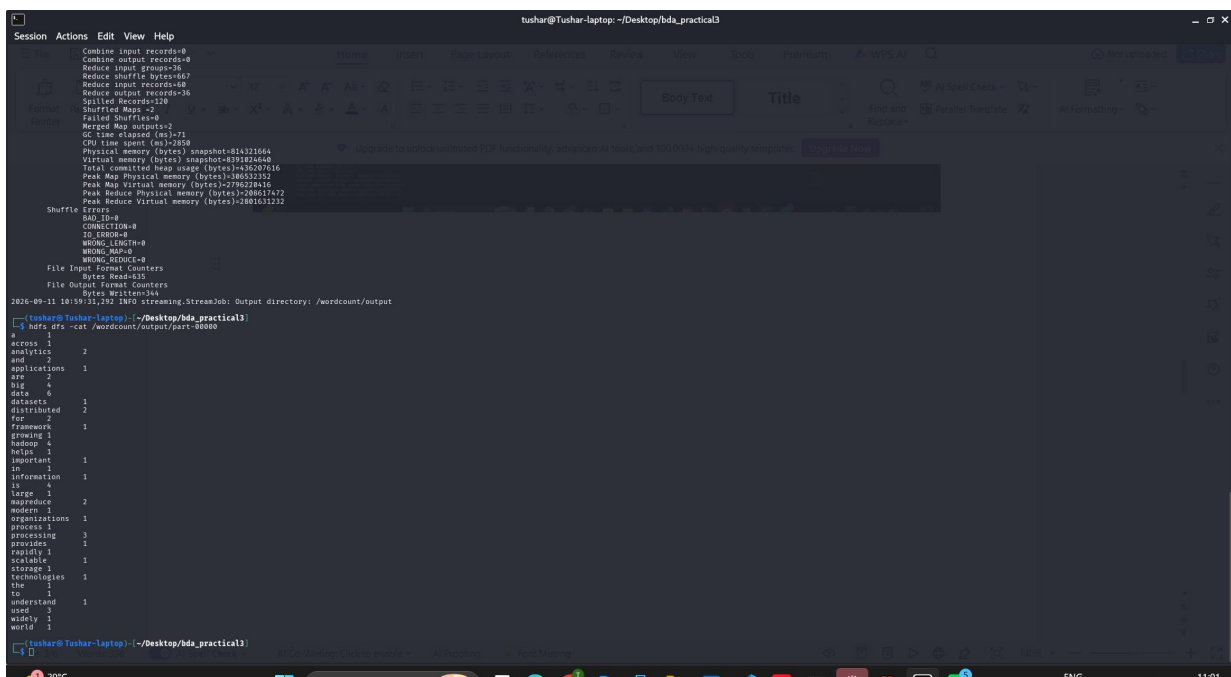
Task 1: Configure the Hadoop environment and upload the given text dataset to HDFS.

```
start-dfs.sh
start-yarn.sh
hdfs dfs -mkdir -p /wordcount/input
hdfs dfs -put news.txt /wordcount/input/
hdfs dfs -cat /wordcount/input/news.txt
```

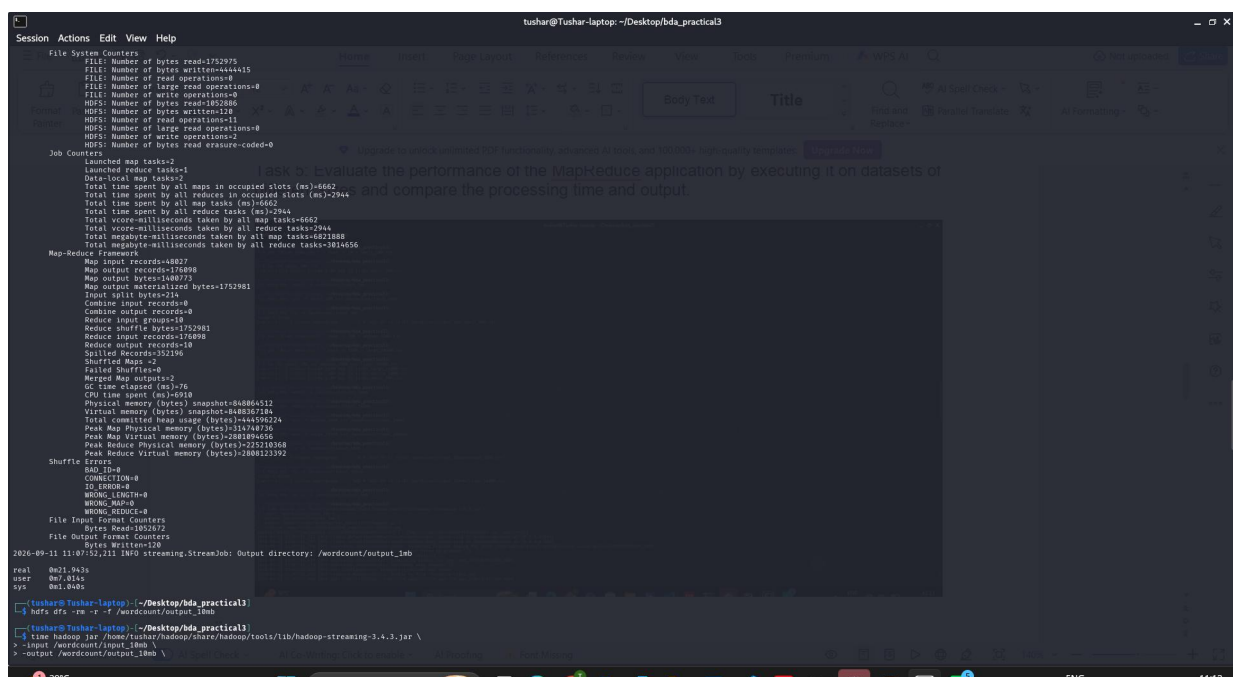
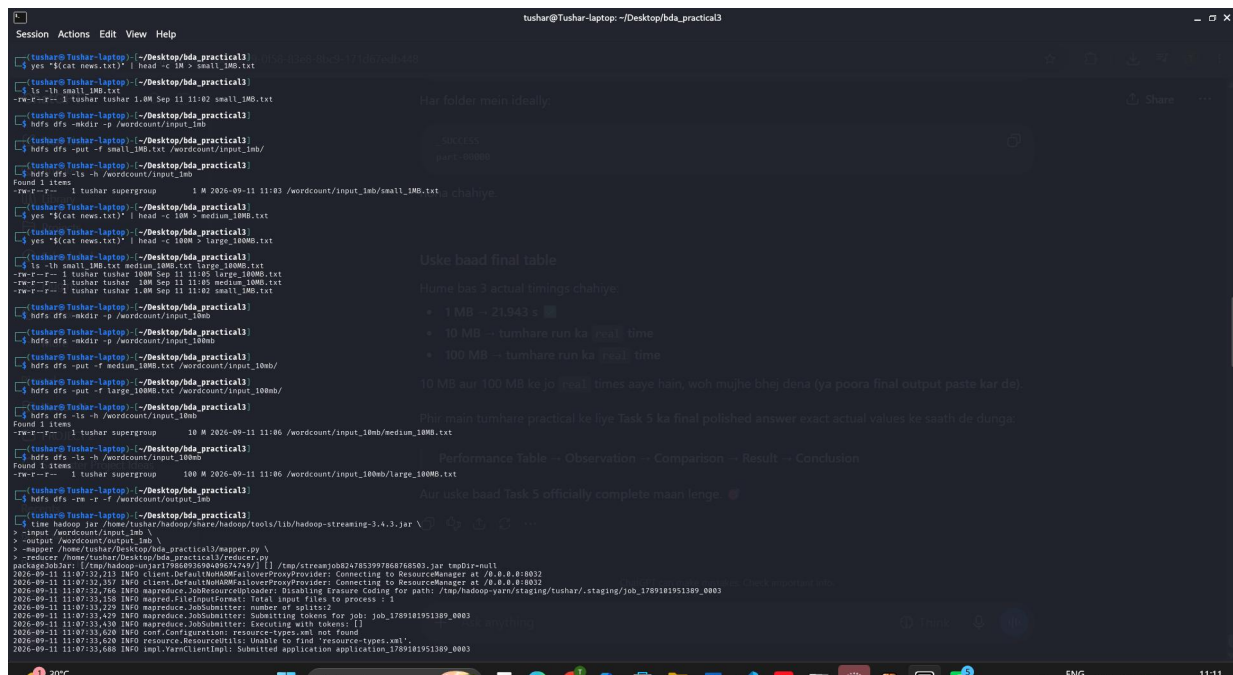
```
nano reducer.py
chmod +x reducer.py
cat news.txt | ./mapper.py | sort | ./reducer.py
```



```
hadoop jar /home/tushar/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.4.3.jar \
-input /wordcount/input \
-output /wordcount/output \
-mapper /home/tushar/Desktop/bda_practical3/mapper.py \
-reducer /home/tushar/Desktop/bda_practical3/reducer.py
```



Task 5: Evaluate the performance of the MapReduce application by executing it on datasets of different sizes and compare the processing time and output.



```

Session Actions Edit View Help
real    0m26.882s
user    0m0.293s
sys     0m1.379s

[tushar@Tushar-Laptop:~/Desktop/bda_practical3]
[hdfs dfs -rm -r -f /wordcount/output_10000]

[tushar@Tushar-Laptop:~/Desktop/bda_practical3]
$ hdfs dfs -rm -r -f /wordcount/output_10000

[tushar@Tushar-Laptop:~/Desktop/bda_practical3]
$ mv hadoop.jar /home/tushar/hadoop/sbin/hadoop/tools/lib/hadoop-streaming-3.4.3.jar \
> -input /wordcount/output_10000 &
> -output /wordcount/output_10000 &
> -mapper /home/tushar/Desktop/bda_practical3/mapper.py \
> -reducer /home/tushar/Desktop/bda_practical3/reducer.py
packageJobJar: [/tmp/hadoop-mapreduce-war-jar2496104806120447/] [] /tmp/streamjob151757083254729663.jar tmpDir=null
2026-09-11 11:10:11.893 INFO Client.DefaultHADOOPFileSystemProvider: Connecting to ResourceManager at 0.0.0.0:8032
2026-09-11 11:10:12.089 INFO Client.DefaultHADOOPFileSystemProvider: Connecting to ResourceManager at (0.0.0.0:8032)
2026-09-11 11:10:12.580 WARN MapperSourceClassLoader: Disabling Feature Codecs for path: /tmp/hadoop-yarn/staging/tushar/.staging/job_1789101951389_0005
2026-09-11 11:10:12.938 INFO Mapper: FileInputFormat: Total input files to process : 1
2026-09-11 11:10:13.423 INFO mapperreduce.JobSubmitter: number of splits=2
2026-09-11 11:10:13.247 INFO mapperreduce.JobSubmitter: Submitting tokens for job: job_1789101951389_0005
2026-09-11 11:10:13.424 INFO mapperreduce.JobSubmitter: Executing with tokens: []
2026-09-11 11:10:13.482 INFO Conf.Configuration: resource-types.xml not found
2026-09-11 11:10:13.482 INFO ConfigurationResourceUtil: Unable to find resource-types.xml..
2026-09-11 11:10:13.568 INFO Impl.VermClientImpl: Submitted application application_1789101951389_0005
2026-09-11 11:10:13.568 INFO ApplicationMaster: The URL for the job: http://tushar-laptop.ticedomain:8080/proxy/application_1789101951389_0005/
2026-09-11 11:10:13.619 INFO mapperreduce.Job: Running job: job_1789101951389_0005
2026-09-11 11:10:13.619 INFO mapperreduce.Job: Job ID = job_1789101951389_0005 running in user mode : false
2026-09-11 11:10:28.748 INFO mapperreduce.Job: map 0% reduce 0%
2026-09-11 11:10:28.748 INFO mapperreduce.Job: map 87% reduce 0%
2026-09-11 11:10:28.748 INFO mapperreduce.Job: map 90% reduce 0%
2026-09-11 11:10:44.821 INFO mapperreduce.Job: map 100% reduce 0%
2026-09-11 11:10:44.821 INFO mapperreduce.Job: map 100% reduce 73%
2026-09-11 11:10:44.821 INFO mapperreduce.Job: map 100% reduce 83%
2026-09-11 11:10:44.821 INFO mapperreduce.Job: map 100% reduce 95%
2026-09-11 11:10:44.821 INFO mapperreduce.Job: map 100% reduce 98%
2026-09-11 11:10:44.821 INFO mapperreduce.Job: map 100% reduce 100%
2026-09-11 11:11:16.558 INFO mapperreduce.Job: Job ID: job_1789101951389_0005 completed successfully
2026-09-11 11:11:16.558 INFO mapperreduce.Job: Counter: 54
File System Counters
FILE: Number of bytes read=385952636
FILE: Number of bytes written=52082919
FILE: Number of read operations=0
FILE: Number of large read operations=0
FILE: Number of write operations=0
HDFS: Number of bytes read=18461914
HDFS: Number of bytes written=138
HDFS: Number of read operations=11
HDFS: Number of large read operations=0
HDFS: Number of write operations=2
HDFS: Number of bytes read erasure-coded=0
Job Counters
Launched map tasks=2
Failed map tasks=0
Killed map tasks=0
Data-local map tasks=2
Total time spent by all maps in occupied slots (ms)=31467
Total time spent by all reduces in occupied slots (ms)=28945
Total time spent by all map tasks (ms)=31467
Total time spent by all reduce tasks (ms)=28945
Total vcore-milliseconds taken by all map tasks=31467
Total vcore-milliseconds taken by all reduce tasks=28945
Total megabyte-milliseconds taken by all map tasks=4510208
Total megabyte-milliseconds taken by all reduce tasks=3812960
Map-Reduce Framework
Map input records=4882639
Map output records=17689674
Map output bytes=14819069
Map output materialized bytes=17529389
Input split bytes=222
Combine input records=0
Combine output records=0
Reduce input groups=10
Reduce shuffle bytes=15256399
Reduce input records=17689674

```

```

Session Actions Edit View Help
Map input records=4882839
Map output records=17489974
Map output bytes=148076945
Map output materialized bytes=175296389
Input split bytes=22
Combine input records=0
Combine output records=0
Reduce input groups=18
Reduce shuffle bytes=175296389
Reduce input records=17489974
Reduce output records=18
Spilled Records=52829822
Shuffled Maps=2
Failed Shuffles=0
Merged Map outputs=2
GC time elapsed (ms)=328
CPU time spent (ms)=70510
Physical memory (bytes) snapshot=1138159136
Virtual memory (bytes) snapshot=848338448
Total committed heap usage (bytes)=489983088
Peak Map Physical memory (bytes)=274532616
Peak Map Virtual memory (bytes)=2881729536
Peak Reduce Physical memory (bytes)=421244928
Peak Reduce Virtual memory (bytes)=2818781184
Shuffle Errors
BAD_ID=0
CONECTID=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0
File Input Format Counters
Bytes Read=184851696
File output Format Counters
Bytes Written=138
2026-09-11 11:11:15,559 INFO streaming.StreamJob: Output directory: /wordcount/output_180mb
real 1m6.932s
user 0m0.911s
sys 0m0.331s

[tushar@Tushar-Laptop: ~/Desktop/hda_practical3]
$ echo "===== 1 MB ====="
dfx dfs -cat /wordcount/output_1mb/part-00000 | wc -l
echo "===== 10 MB ====="
dfx dfs -cat /wordcount/output_10mb/part-00000 | wc -l
echo "===== 100 MB ====="
dfx dfs -cat /wordcount/output_100mb/part-00000 | wc -l
echo "===== 1 MB ====="
dfx dfs -cat /wordcount/output_1mb/part-00000 | wc -l
echo "===== 10 MB ====="
dfx dfs -cat /wordcount/output_10mb/part-00000 | wc -l
echo "===== 100 MB ====="
dfx dfs -cat /wordcount/output_100mb/part-00000 | wc -l

[tushar@Tushar-Laptop: ~/Desktop/hda_practical3]
$ hdfs dfs -ls /wordcount/output_1mb
hdfs dfs -ls /wordcount/output_10mb
hdfs dfs -ls /wordcount/output_100mb
Found 2 items:
-rw-r--r-- 1 tushar supergroup 0 2026-09-11 11:07 /wordcount/output_1mb/SUCCESS
-rw-r--r-- 1 tushar supergroup 120 2026-09-11 11:07 /wordcount/output_1mb/part-00000
Found 2 items:
-rw-r--r-- 1 tushar supergroup 0 2026-09-11 11:09 /wordcount/output_10mb/SUCCESS
-rw-r--r-- 1 tushar supergroup 130 2026-09-11 11:09 /wordcount/output_10mb/part-00000
Found 2 items:
-rw-r--r-- 1 tushar supergroup 0 2026-09-11 11:11 /wordcount/output_100mb/SUCCESS
-rw-r--r-- 1 tushar supergroup 130 2026-09-11 11:11 /wordcount/output_100mb/part-00000

[tushar@Tushar-Laptop: ~/Desktop/hda_practical3]

```

Key Questions /Analysis:'

Q1. What is the role of Mapper and Reducer in the Word Count application?

Mapper: Reads input data, extracts words, and generates intermediate key-value pairs:

Hadoop Hadoop data

Mapper output:

(Hadoop, 1)

(Hadoop, 1)

(data, 1)

Reducer: Receives all values associated with the same key and calculates their total:

(Hadoop, [1, 1])

Output:

(Hadoop, 2)

Q2. How are intermediate key-value pairs generated and processed?

The Mapper reads each line, separates it into words, converts them to lowercase, removes punctuation, and produces:

(word, 1)

During the **Shuffle and Sort phase**, Hadoop automatically groups identical keys:

data → [1, 1, 1, 1]

hadoop → [1, 1, 1]

Then the Reducer calculates the sum.

Q3. How does HDFS store and distribute the input dataset?

HDFS divides large files into blocks and stores those blocks across DataNodes. Blocks can also be replicated for fault tolerance. Hadoop attempts to run computation close to where data is stored, known as **data locality**, reducing network transfer.

Q4. What advantages does MapReduce offer for processing large datasets?

- Processes very large datasets.
- Supports distributed processing.
- Provides scalability.
- Offers fault tolerance.

- Uses data locality.
- Can process multiple files together.
- Automatically handles shuffle, sort, and task coordination.

Q5. How does the execution time vary with dataset size?

However, the relationship may not be perfectly linear because Hadoop includes overhead for:

- Job initialization
- Task scheduling
- Shuffle and sort
- Disk I/O
- HDFS operations

For small datasets, this overhead can be significant.

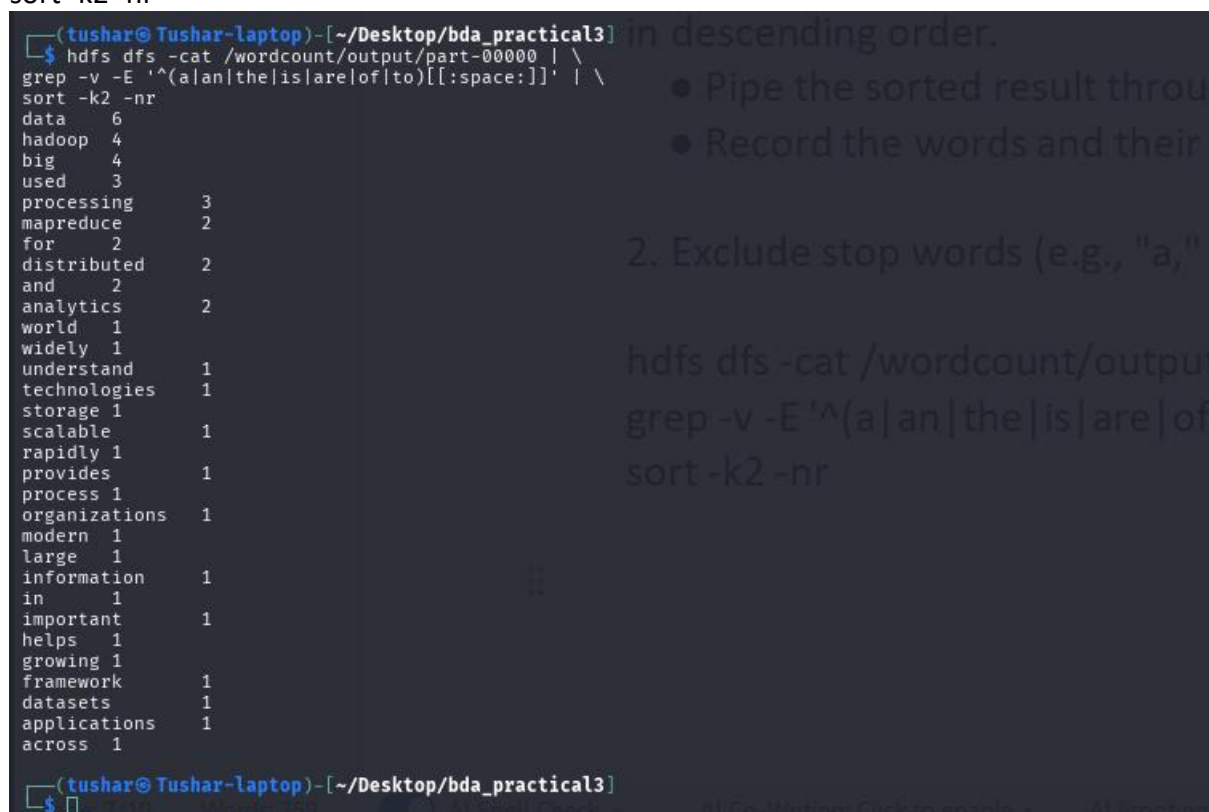
Supplementary Problems:

1. Find the top 10 most frequently occurring words.

- Run the WordCount job to produce the part-r-00000 output file in HDFS.
- Use `hdfs dfs -cat` on the output file and pipe it through `sort -k2,2 -nr` to sort by frequency in descending order.
- Pipe the sorted result through `head -10` to extract the ten highest-frequency words.
- Record the words and their counts as the final top-10 list.

2. Exclude stop words (e.g., "a," "an," "the," "is") from the analysis.

```
hdfs dfs -cat /wordcount/output/part-00000 | \
grep -v -E '(a|an|the|is|are|of|to)[[:space:]]' | \
sort -k2 -nr
```



```
(tushar@Tushar-laptop)-[~/Desktop/bda_practical3]
$ hdfs dfs -cat /wordcount/output/part-00000 | \
grep -v -E '(a|an|the|is|are|of|to)[[:space:]]' | \
sort -k2 -nr
data 6
hadoop 4
big 4
used 3
processing 3
mapreduce 2
for 2
distributed 2
and 2
analytics 2
world 1
widely 1
understand 1
technologies 1
storage 1
scalable 1
rapidly 1
provides 1
process 1
organizations 1
modern 1
large 1
information 1
in 1
important 1
helps 1
growing 1
framework 1
datasets 1
applications 1
across 1
```

3. Compare word frequency results for multiple input files.

- Upload all the input files (e.g., input1.txt, input2.txt, ...) into the same HDFS input directory.
- Run the WordCount job once with FileInputFormat pointed at the whole directory — Hadoop automatically creates one split per file/block and processes them all in the same job.
- The Reducer aggregates counts across every file automatically, since counts are grouped by word key regardless of which file/Mapper produced them.
- To compare files individually, run the job separately on each file's own input directory and diff the resulting part-r-00000 outputs, or emit the source file name as part of the key for a per-file breakdown.

Key Skills to be Addressed:

Hadoop Ecosystem Fundamentals | HDFS Operations | MapReduce Programming | Distributed Data Processing | Data Analysis and Interpretation

Tools / Technology used:

Hadoop 3.x | HDFS | Hadoop Streaming | Python 3 | Linux / Ubuntu Environment.

Conclusion

The Word Count application was successfully implemented using Hadoop MapReduce and Python.

The Mapper generated key-value pairs in the form of (word, 1), while the Reducer calculated the total frequency of each word.

The MapReduce job was executed successfully on HDFS and the most frequent words were identified after removing stop words.

The application was also tested on datasets of different sizes to compare processing performance.

The experiment demonstrated how MapReduce can efficiently process large-scale text data using distributed processing.

